

SESIÓN 2: DATA WRANGLING Y PIVOTS

## Data Wrangling y Pivots

Fausto Jácome Pérez  
FLACSO Ecuador

*Tengo los datos en memoria. ¿Cómo los preparo para responder una pregunta sobre desnutrición infantil?*

## ¿Dónde estamos?

1

SESIÓN ANTERIOR

Cargamos `f1_personas`. Vimos el schema, los tipos inferidos, los missings. Terminamos con una pregunta pendiente: ¿por qué `area` es `Float64`?

2

ESTA SESIÓN

Corregir tipos. Filtrar. Recodificar. Reshapear. Todas las operaciones que ocurren antes de calcular cualquier indicador.

3

PRÓXIMA SESIÓN

Funciones de ventana: calcular indicadores dentro de grupos sin perder filas.

El objetivo de hoy: dejar `f1_personas` lista para calcular prevalencia de desnutrición crónica. Y entender por qué la vacunación de rotavirus *obliga* a reshapear.

## Paso 0: corregir tipos

Antes de filtrar o calcular, los tipos tienen que ser los correctos

# El problema de los tipos mal inferidos

Cuando Polars lee un CSV, infiere los tipos columna por columna. Lo hace bien para `fexp` (es genuinamente `Float64` ). Pero se equivoca con columnas que tienen *códigos numéricos*:

| Variable             | Tipo inferido                             | Tipo correcto            | Por qué importa                      |
|----------------------|-------------------------------------------|--------------------------|--------------------------------------|
| <code>area</code>    | <code>Float64</code>                      | <code>Categorical</code> | Urbano=1, Rural=2 no son magnitudes  |
| <code>region</code>  | <code>Float64</code>                      | <code>Categorical</code> | Costa=1, Sierra=2, Amazonia=3        |
| <code>prov</code>    | <code>Float64</code> o <code>Int64</code> | <code>String</code>      | Código de provincia, no número       |
| <code>id_upm</code>  | <code>Int64</code>                        | <code>String</code>      | Es un identificador, no una cantidad |
| <code>f1_s1_2</code> | <code>Int64</code>                        | <code>Categorical</code> | Sexo: 1=Hombre, 2=Mujer              |

Un `Float64` donde debería haber una categoría no genera error - simplemente devuelve resultados sin sentido. Promediar `area` da 1.38. Nadie lo advierte.

## Cast con `.with_columns()` - por nombre explícito

La forma más clara para estudiantes: una sola llamada `.with_columns()` con una lista de expresiones `cast`:

```
import polars as pl
df = df.with_columns([
    # Variables geográficas y de diseño muestral
    pl.col("area").cast(pl.Categorical),
    pl.col("region").cast(pl.Categorical),
    pl.col("estrato").cast(pl.Categorical),
    pl.col("parr_pri").cast(pl.Categorical),

    # Identificadores: son códigos, no cantidades
    pl.col("id_upm").cast(pl.String),
    pl.col("id_viv").cast(pl.String),
    pl.col("id_hogar").cast(pl.String),
    pl.col("id_per").cast(pl.String),

    # Variables sociodemográficas
    pl.col("f1_s1_2").cast(pl.Categorical), # sexo
    pl.col("prov").cast(pl.String),         # código provincia
])
```

## Selectores - referencia rápida

Los selectores ( `cs` ) son expresiones que describen *grupos de columnas* por tipo, nombre o patrón. Se importan aparte:

```
import polars.selectors as cs
```

### Por tipo de dato

```
cs.numeric()      # Int*, Float*, UInt*
cs.string()       # Utf8 / String
cs.boolean()      # Bool
cs.temporal()     # Date, Datetime, Duration, Time
cs.categorical()  # Categorical
```

### Por nombre o patrón

```
cs.starts_with("f1_s5_") # acepta multiples prefijos
cs.ends_with("_id", "_cod")
cs.contains("monto", "valor")
cs.by_name("edad", "sexo") # equivalente a pl.col() pero componible
```

### Condiciones compuestas

```
# Numéricas excepto identificadores
cs.numeric() & ~cs.ends_with("_id")
# Variables de monto o de fecha
cs.contains("monto") | cs.temporal()
# Todo excepto strings
~cs.string()
```

### Convertir a expresión

```
# Necesario cuando se mezcla con pl.col()
cs.numeric().as_expr()
```

## Selectores en `.with_columns()` - caso ENDI

---

El caso más útil en encuestas: aplicar la misma transformación a un grupo de columnas.

### Caso 1 - Reemplazar códigos de no respuesta en todas las columnas numéricas de una sección

```
# En la ENDI, 88 = "No sabe", 99 = "No responde", 9999 = omisión
df = df.with_columns(
    cs.starts_with("f1_s1_").replace({88: None, 99: None, 9999: None})
)
```

### Caso 2 - Cast de todos los identificadores en una sola línea

```
# En lugar de listar id_upm, id_viv, id_hogar, id_per uno por uno
df = df.with_columns(
    cs.by_name("id_upm", "id_viv", "id_hogar", "id_per", "id_mef").cast(pl.String)
)
```

# Filtros y selección

`.filter()`, `.select()`, `.drop()` y selectores



## `.filter()` - filas que cumplen una condición

La sintaxis básica es siempre `pl.col("nombre") <operador> <valor>`:

```
# Menores de 5 años (< 1826 días) - población objetivo de desnutrición crónica
menores5 = df.filter(pl.col("edaddias") < 1826)

# Menores de 2 años (< 730 días) - grupo objetivo del Decreto Presidencial 1211
menores2 = df.filter(pl.col("edaddias") < 730)

# Solo área urbana (código 1)
urbanos = df.filter(pl.col("area") == pl.lit("1"))

# Condiciones múltiples con & (and) y | (or)
urbanos_menores5 = df.filter(
    (pl.col("area") == pl.lit("1")) &
    (pl.col("edaddias") < 1826)
)
```

Con `Categorical`, comparar contra strings requiere `pl.lit()`. Sin él, Polars puede lanzar un error de tipos o comparar de forma inesperada.

## `.select()` y `.drop()` - columnas que necesito

`.select()` elige columnas. `.drop()` descarta. Ambas admiten selectores:

```
import polars.selectors as cs

# Selección manual: las columnas clave para desnutrición
df_trabajo = df.select([
    "id_per", "id_hogar",
    "area", "region", "prov",
    "fexp", "estrato",
    "f1_s1_2",      # sexo
    "edaddias",     # calculada en sesión 1
    "dcronica",     # indicador precalculado
])

# Con selectores: quedarme con IDs + categóricas + floats
df_trabajo = df.select(
    cs.by_name("id_per", "id_hogar", "fexp", "estrato") |
    cs.categorical() |
    cs.by_name("edaddias", "dcronica")
)
```

## Piedra Rosetta - Cast y Filtros

### Python · Polars

```
# Cast
df = df.with_columns([
    pl.col("area")
        .cast(pl.Categorical),
    pl.col("region")
        .cast(pl.Categorical),
])

# Filtro
menores5 = df.filter(
    pl.col("edaddias") < 1826
)
```

### R · dplyr

```
library(dplyr)

# Cast
df <- df %>%
  mutate(
    area = as.factor(area),
    region = as.factor(region)
  )

# Filtro
menores5 <- df %>%
  filter(edaddias < 1826)
```

### SQL · PostgreSQL

```
-- Cast (al crear la tabla)
ALTER TABLE personas
  ALTER COLUMN area
  TYPE VARCHAR;

-- Filtro
SELECT *
FROM personas
WHERE edaddias < 1826;
```

# Ejercicio 1

Identificar y corregir los tipos de las variables restantes

## Ejercicio 1 - Cast de variables pendientes

**Objetivo:** corregir los tipos de las variables que dejamos pendientes en la sesión 1.

Usando el diccionario de la ENDI ( `Diccionario_variables_ENDI_R2.xlsx` , hoja `f1_personas` ):

1. Identifiquen 3 variables adicionales que tienen tipo incorrecto (distintas a las que ya corregimos)
2. Para cada una, digan cuál debería ser el tipo correcto y por qué
3. Apliquen el cast con `.with_columns()`
4. Verifiquen el resultado con `df.schema`

**Pistas:** miren las variables de la Sección 1 del formulario (información del miembro del hogar) y la Sección 2 (actividades económicas). ¿Tiene sentido que `f1_s1_3` (relación de parentesco) sea `Int64` ?

No toquen `fexp` , `fexp_lm` , `fexp_di` - esos `Float64` son correctos.

# Recodificación

`.with_columns()`, `pl.when().then().otherwise()` y mapeo de etiquetas

## ¿Qué es recodificar?

Recodificar es transformar los valores de una columna existente en algo más útil para el análisis. Hay tres casos típicos:

### Mapear etiquetas

Pasar de código numérico a texto legible.

1 → "Urbana"  
2 → "Rural"

Necesario porque la ENDI entrega códigos, no etiquetas.

### Crear variables derivadas

Construir una variable nueva a partir de otras.

edaddias → grupos de edad: "0-23 meses", "24-59 meses"

Necesario para desagregaciones analíticas.

### Corregir o limpiar

Reemplazar códigos de no respuesta por `null`.

88 , 99 , 9999 → null

Necesario antes de cualquier cálculo.

En Polars, todas estas operaciones usan `pl.when().then().otherwise()` dentro de `.with_columns()`.

## Mapecto de etiquetas con `pl.when().then().otherwise()`

Caso concreto: `area` (1 = Urbana, 2 = Rural) y `region`:

```
df = df.with_columns([\n\n    # Etiqueta de area\n    pl.when(pl.col("area") == pl.lit("1")).then(pl.lit("Urbana"))\n      .when(pl.col("area") == pl.lit("2")).then(pl.lit("Rural"))\n      .otherwise(pl.lit(None))\n      .alias("area_etiq"),\n\n    # Etiqueta de region (Costa=1, Sierra=2, Amazonia=3)\n    pl.when(pl.col("region") == pl.lit("1")).then(pl.lit("Costa"))\n      .when(pl.col("region") == pl.lit("2")).then(pl.lit("Sierra"))\n      .when(pl.col("region") == pl.lit("3")).then(pl.lit("Amazonia"))\n      .otherwise(pl.lit(None))\n      .alias("region_etiq"),\n\n])
```

`.alias("nombre")` crea la columna con ese nombre. Sin alias, Polars usa el nombre de la columna original y **sobreescribe** la columna.



## `pl.Categorical` vs `pl.Enum` - cuándo usar cada uno

Una columna de **texto** creada con `pl.when()` tiene tipo `pl.String`. Si los valores son un conjunto cerrado y finito, Polars puede trabajar con ellos de forma más eficiente usando `pl.Categorical` o `pl.Enum`.

### `pl.Categorical`

Polars construye la lista de categorías automáticamente al leer los datos. No hay un orden definido.

Usar cuando:

- Las categorías no tienen jerarquía natural
- No se conocen de antemano todos los valores posibles
- Se trabaja con variables nominales: `area`, `region`, `prov`, `sexo`

```
pl.col("area_etiq").cast(pl.Categorical)
```

### `pl.Enum`

Las categorías se declaran explícitamente y en orden. Polars las trata como una lista cerrada y ordenada.

Usar cuando:

- Se trabaja con variables ordinales: nivel educativo, quintil, grupo de edad
- Se quiere garantizar que no aparezcan valores inesperados

```
pl.col("grupo_edad").cast(pl.Enum(["0-23 meses", "24-59 meses"]))
```

`pl.Enum` da error si encuentra un valor que no está en la lista declarada. Actúa como validación automática de los datos.

## pl.Categorical y pl.Enum - ejemplos con la ENDI

```
from polars import Enum
# Variables nominales: sin orden, categorías conocidas
df = df.with_columns([
    pl.col("area_etiq").cast(pl.Categorical),    # Urbana / Rural
    pl.col("region_etiq").cast(pl.Categorical),  # Costa / Sierra / Amazonía
    pl.col("prov").cast(pl.Categorical),         # código de provincia
])
# Variable ordinal: grupo de edad con orden explícito
orden_edad = ["0-23 meses", "24-59 meses"]
df = df.with_columns(
    pl.col("grupo_edad").cast(pl.Enum(orden_edad))
)
# Verificar que el orden se respeta al ordenar
df.filter(pl.col("grupo_edad").is_not_null()) \
    .sort("grupo_edad") \
    ["grupo_edad"].unique()
# Resultado: ["0-23 meses", "24-59 meses"] - en ese orden, no alfabético
```

Sin `pl.Enum`, al ordenar una columna de strings Polars usaría orden alfabético: `"24-59 meses"` quedaría antes que `"0-23 meses"`. Con `pl.Enum` el orden es el que se declaró.

## Crear variables derivadas - grupos de edad

Para el indicador de desnutricion cronica, el INEC trabaja con dos grupos: menores de 2 anos y menores de 5. Construimos la variable de grupo:

```
df = df.with_columns(  
    pl.when(  
        (pl.col("edaddias") >= 0) & (pl.col("edaddias") < 730)  
    ).then(pl.lit("0-23 meses"))  
    .when(  
        (pl.col("edaddias") >= 730) & (pl.col("edaddias") < 1826)  
    ).then(pl.lit("24-59 meses"))  
    .otherwise(pl.lit(None))  
    .alias("grupo_edad")  
)  
  
# Verificar  
df.filter(  
    pl.col("edaddias").is_not_null()
```

730 dias =  $365.25 \times 2$ . 1826 dias =  $365.25 \times 5$ . El decimal importa cuando las edades se calculan en dias exactos - es la metodologia del Manual WHO Anthro.

## Piedra Rosetta - Recodificacion

```
# Crear etiqueta de area
df = df.with_columns(
    pl.when(pl.col("area") == pl.lit("1"))
        .then(pl.lit("Urbana"))
        .when(pl.col("area") == pl.lit("2"))
        .then(pl.lit("Rural"))
        .otherwise(pl.lit(None))
        .alias("area_etiq")
)

# Crear grupo de edad
df = df.with_columns(
    pl.when(pl.col("edaddias") < 730)
        .then(pl.lit("0-23 meses"))
        .when(pl.col("edaddias") < 1826)
        .then(pl.lit("24-59 meses"))
        .otherwise(pl.lit(None))
        .alias("grupo_edad")
)
```

```
library(dplyr)
library(sjlabelled)

# En R, la ENDI ya viene con etiquetas
# as_label() las convierte directamente
df <- df %>%
    mutate(area = as_label(area))

# O con case_when (equivalente manual)
df <- df %>%
    mutate(
        area_etiq = case_when(
            area == 1 ~ "Urbana",
            area == 2 ~ "Rural",
            TRUE      ~ NA_character_
        ),
        grupo_edad = case_when(
            edaddias < 730 ~ "0-23 meses",
            edaddias < 1826 ~ "24-59 meses",
            TRUE          ~ NA_character_
        )
    )
```

## Ejercicio 2

Construir la etiqueta de region y filtrar menores de 2 anos

## Ejercicio 2 - Region y grupo objetivo del decreto

**Objetivo:** construir una variable etiquetada para `region` y filtrar el grupo objetivo del Decreto Presidencial 1211.

1. Creen la variable `region_etiq` con las etiquetas: Costa, Sierra, Amazonia
2. Filtren el subconjunto de menores de 2 años ( `edaddias < 730` )
3. Sobre ese subconjunto, cuenten cuántos casos hay por region usando `.group_by("region_etiq").agg(pl.len())`
4. Respondan: ¿en qué region hay mas menores de 2 años en la muestra?

**Conexion con el indicador:** el Decreto Presidencial 1211 (2020) fijo como meta reducir la desnutricion cronica en menores de 24 meses. Por eso el INEC reporta `dcronica2_5` como variable separada para ese subgrupo.

`dcronica` cubre menores de 5 anos. `dcronica2_5` cubre menores de 2 anos. Son variables precalculadas en la tabla - no hay que construirlas desde cero en este tramo.

# Reshaping: Long vs Wide

La forma de la tabla determina las preguntas que puedes responder

# El problema de la tabla MEF

La tabla `f2_mef` registra informacion de mujeres en edad fertil. Cada mujer puede tener varios hijos. El INEC guardo los datos en formato **wide**: cada hijo es un conjunto de columnas.

## Estructura wide (lo que tiene la tabla):

| id_mef | f2_s2_235_a_01 | f2_s2_235_b_dia_01 | f2_s2_235_a_02 | f2_s2_235_b_dia_02 | ... |
|--------|----------------|--------------------|----------------|--------------------|-----|
| M001   | 1 (Hombre)     | 15                 | 2 (Mujer)      | 3                  | ... |
| M002   | 2 (Mujer)      | 22                 | null           | null               | ... |

**El problema:** para calcular la edad de vacunacion de cada hijo, necesito una fila por hijo, no una columna por hijo.

En formato wide no se puede filtrar *"todos los hijos menores de 1 ano"* con un solo `.filter()` . Habria que escribir una condicion por cada columna de hijo. Con hasta 10 hijos por mujer, son 10 condiciones.



## Wide a Long - concepto

**Wide:** cada observacion es una fila. Las categorias de una variable se distribuyen en columnas.

```
id_mef | hijo_01_sexo | hijo_02_sexo
M001   | Hombre         | Mujer
M002   | Mujer          | null
```

Util para: presentacion, tablas cruzadas, cuando los atributos son fijos y pocos.

**Long:** cada combinacion (unidad, categoria) es una fila. Una columna identifica la categoria, otra tiene el valor.

```
id_mef | ord_hijo | sexo
M001   | 01       | Hombre
M001   | 02       | Mujer
M002   | 01       | Mujer
```

Util para: analisis, filtros, calculos, graficos. Es la forma **tidy** de los datos.

La mayoría de las operaciones de analisis esperan datos en formato **long**. El formato wide es para leer, el long es para calcular.

## `.unpivot()` - de wide a long en Polars

La sintaxis oficial de Polars para pasar de wide a long es `.unpivot()` :

```
# Seleccionamos solo las columnas de hijos (sexo y fecha nacimiento)
cols_hijos = [c for c in df_mef.columns
               if c.startswith("f2_s2_235_")]

df_hijos = df_mef.select(["id_mef"] + cols_hijos)

# Wide a Long
df_long = df_hijos.unpivot(
    on=cols_hijos,          # columnas que se van a "apilar"
    index=["id_mef"],       # columnas que se mantienen como identificador
    variable_name="campo",  # nombre de la columna que tendra el nombre de la variable
    value_name="valor",     # nombre de la columna que tendra el valor
)
```

El resultado tiene `n_filas_original × n_columnas_apiladas` filas. Si la tabla original tiene 19.846 mefs y 50 columnas de hijos, el resultado tiene ~992.300 filas.

## `.pivot()` - de long a wide en Polars

La operacion inversa: colapsar una columna de categorias en columnas separadas.

```
# Ejemplo: de long a wide, una columna por tipo de vacuna
df_wide = df_vacunas_long.pivot(
    on="tipo_vacuna",      # columna cuyos valores se convierten en nombres de columna
    index="id_hijo_ord",   # columna(s) que identifican la unidad de analisis
    values="aplicada",     # columna con los valores a distribuir
    aggregate_function="first" # si hay duplicados, tomar el primero
)
```

**Cuidado con los duplicados:** si hay mas de un valor por combinacion (index, on), Polars aplica la funcion de agregacion. Si no se especifica, lanza un error. Lo mas comun es `"first"` o `"sum"`.

## El flujo completo para vacunacion de rotavirus

La ficha metodologica del INEC hace exactamente esto: wide → long → calcular → long → wide

```
# 1. Seleccionar columnas de hijos de la MEF
cols_hijos = [c for c in df_mef.columns if "f2_s2_235_" in c]
df_hijos = df_mef.select(["id_mef"] + cols_hijos)

# 2. Wide a Long (una fila por hijo)
df_hijos_long = df_hijos.unpivot(
    on=cols_hijos, index=["id_mef"],
    variable_name="campo", value_name="valor"
)

# 3. Separar el nombre del campo para extraer orden del hijo
# (esto lo trabajamos en detalle en el ejercicio)

# 4. Calcular edad de vacunacion en dias por cada dosis

# 5. Long a wide (volver a una fila por hijo con columnas por dosis)
df_vacunas_wide = df_vacunas_long.pivot(
    on="dosis", index="id_hijo_ord", values="edad_dias_vac"
)
```

# Piedra Rosetta - Reshaping

## Python · Polars

```
# Wide a Long
df_long = df_wide.unpivot(
    on=cols_variables,
    index=["id_mef"],
    variable_name="campo",
    value_name="valor"
)

# Long a wide
df_wide2 = df_long.pivot(
    on="campo",
    index=["id_mef"],
    values="valor",
    aggregate_function="first"
)
```

## R · tidyr

```
library(tidyr)

# Wide a Long
df_long <- df_wide %>%
  pivot_longer(
    cols = starts_with("f2_s2_235_"),
    names_to = "campo",
    values_to = "valor"
)

# Long a wide
df_wide2 <- df_long %>%
  pivot_wider(
    id_cols = id_mef,
    names_from = campo,
    values_from = valor
  )
```

## Ejercicio 3

Primer paso del indicador de vacunacion de rotavirus

## Ejercicio 3 - De wide a long en f2\_mef

**Objetivo:** aplicar `.unpivot()` para obtener una fila por hijo a partir de la tabla `f2_mef`.

### Pasos:

1. Carguen `f2_mef` (ya lo hicieron en la sesion 1 si siguieron el ejercicio)
2. Identifiquen cuántas columnas empiezan con `f2_s2_235_` - esas son los campos de hijos
3. Apliquen `.unpivot()` sobre esas columnas, manteniendo como index: `id_mef`, `id_upm`, `estrato`, `fexp`
4. Filtren las filas donde `valor` no es nulo (esas corresponden a hijos registrados)
5. ¿Cuántas filas tiene el resultado? ¿Tiene sentido comparado con el numero de mefs?

**Pregunta de cierre:** ¿por qué no podriamos calcular la edad de vacunacion de rotavirus directamente sobre la tabla wide sin hacer el unpivot primero?

# ¿Qué aprendimos hoy?

## Conceptos

- Cast: por que los tipos mal inferidos son errores silenciosos
- Filtros con condiciones multiples ( `&` , `|` )
- Selectores ( `cs` ) para operar por tipo o nombre
- Recodificacion: etiquetas, variables derivadas, limpieza de no respuesta
- Long vs wide: cuando usar cada formato

## En Polars

- `.with_columns([...])` con listas de expresiones
- `.cast(pl.Categorical)` , `.cast(pl.String)`
- `cs.by_name()` , `cs.by_dtype()` , `cs.categorical()`
- `.filter()` con `pl.col()` y `pl.lit()`
- `.select()` y `.drop()` con selectores
- `pl.when().then().otherwise().alias()`
- `.unpivot()` y `.pivot()`

**Tarea (para el lunes):** a partir de `f1_personas` , aplicar los casts necesarios, filtrar menores de 5 anos, recodificar `area` y `region` , y calcular la prevalencia de `dcronica` por area como promedio simple. El resultado debe ser una tabla de dos filas.



## Tarea - Detalle

**Hay tarea para esta sesion.** Se entrega como celda adicional en el notebook de la sesion 2.

### Pasos requeridos:

1. Cargar `f1_personas` y aplicar todos los casts de tipos correctos (al menos area, region, prov, identificadores)
2. Filtrar menores de 5 anos ( `edaddias < 1826` ) y que `dcronica` no sea nula
3. Crear la variable `area_etiq` con etiquetas "Urbana" / "Rural"
4. Calcular la prevalencia de `dcronica` por area:

```
df.groupby("area_etiq").agg(pl.mean("dcronica").alias("prevalencia"))
```

5. Interpretar el resultado: ¿en qué area es mayor la desnutricion cronica?

**Lo que NO se pide todavia:** aplicar el factor de expansion `fexp` . El promedio simple es una aproximacion - las cifras oficiales del INEC usan el diseno muestral completo, que veremos en la sesion 4.

**Proximamente:** funciones de ventana - calcular indicadores dentro de grupos sin perder las filas originales.